Web Application Penetration Testing

Using OWASP Juice Shop & Burp Suite

OWASP Top 10 Vulnerability Assessment Report

Abhay Singh Rathore
Cybersecurity Intern
Zaalima Learnings

May 21, 2026

Project Type: Web Application Penetration Testing
Target Application: OWASP Juice Shop (Vulnerable by Design)
Testing Framework: OWASP Top 10 (2021)
Tools Used: Kali Linux, Burp Suite, Docker, Firefox
Classification: Educational / Controlled Environment

Contents

1 Executive Summary

This report documents the web application penetration testing performed on **OWASP Juice Shop**, an intentionally vulnerable web application, as part of the Zaalima Learnings cybersecurity internship program. The testing was conducted over a four-week structured period following the OWASP Top 10 (2021) vulnerability framework.

The assessment identified and successfully exploited the following vulnerability categories:

S.No.	Vulnerability	Severity	OWASP Category
1	DOM-Based Cross-Site Scripting (XSS)	**High**	A03:2021 - Injection
2	Bonus Payload XSS	**Medium**	A03:2021 - Injection
3	Broken Authentication (Password Strength)	**High**	A07:2021 - Auth Failures
4	Information Disclosure (Privacy Policy)	**Medium**	A02:2021 - Cryptographic
5	Business Logic Vulnerability (Chatbot)	**Medium**	A04:2021 - Insecure Design

All five challenges were successfully solved and confirmed by the application's built-in scoring system.

2 Introduction

Web applications are one of the most targeted attack surfaces in modern cybersecurity. Vulnerabilities such as Cross-Site Scripting (XSS), Broken Authentication, Injection attacks, and Information Disclosure can compromise sensitive user data and application functionality, leading to significant financial and reputational damage.

This project focuses on performing penetration testing on **OWASP Juice Shop** using Kali Linux and Burp Suite. The testing process follows OWASP Top 10 vulnerability categories and demonstrates real-world attack scenarios in a controlled, legal, and ethical environment.

OWASP Juice Shop is a modern, full-stack JavaScript web application specifically designed to teach and practice offensive security skills. It includes challenges across all OWASP Top 10 categories and provides instant feedback on successful exploitation.

3 Objectives

The primary objectives of this penetration testing engagement are:

1. To understand and apply web application penetration testing concepts.
2. To identify and exploit vulnerabilities present in OWASP Juice Shop.
3. To perform testing using Burp Suite and browser-based attack vectors.

4. To analyze HTTP requests and responses for security weaknesses.
5. To gain practical, hands-on knowledge of OWASP Top 10 vulnerabilities.
6. To document findings in a professional penetration testing report format.
7. To develop skills relevant to a career in offensive security and ethical hacking.

4 Project Timeline (Week-wise Plan)

This project was structured as a four-week engagement following the Zaalima Learnings internship curriculum:

Week	Tasks and Activities
Week 1	• Set up Kali Linux environment (local or VM). • Install and configure Docker. • Deploy OWASP Juice Shop container. • Configure Burp Suite as a proxy and set up Firefox. • Explore the application structure and understand the attack surface. • Solve initial reconnaissance challenges.
Week 2	• Perform DOM-Based XSS attack (basic iframe payload). • Execute Bonus Payload XSS (SoundCloud iframe). • Analyze HTTP traffic with Burp Suite Proxy. • Document payloads, results, and challenge completions. • Mid-project review: Basic attacks confirmed and documented.
Week 3	• Perform Broken Authentication testing (Password Strength challenge). • Identify Information Disclosure via Privacy Policy page. • Exploit Business Logic Vulnerability using the chatbot. • Capture and analyze all related HTTP requests in Burp Suite. • Refine and expand documentation of each vulnerability.

Week	Tasks and Activities
Week 4	• Perform final review of all identified vulnerabilities. • Fine-tune descriptions and evidence screenshots. • Write mitigation recommendations for each vulnerability. • Compile the full penetration testing report. • Prepare deliverables: report, security checklist, and presentation.

Mid-Project Review (End of Week 2): Environment fully set up. XSS attacks completed.
Final Review (End of Week 4): All 5 challenges solved. Full report submitted.

5 Tools and Technologies Used

Tool	Version/Type	Purpose
Kali Linux	Rolling Release	Penetration Testing OS
OWASP Juice Shop	Docker (latest)	Vulnerable Target Application
Docker	Community Edition	Container Runtime for Juice Shop
Burp Suite	Community Edition	HTTP Proxy and Request Analysis
Firefox Browser	ESR	Web Application Access
PortSwigger API	Built-in	Request Interception

6 Environment Setup

6.1 Step 1: Installing Docker on Kali Linux

```
sudo apt update
sudo apt install docker.io -y
```

Listing 1: Installing Docker

6.2 Step 2: Starting Docker Service

```
sudo systemctl start docker
sudo systemctl enable docker
# Verify Docker is running
sudo systemctl status docker
```

Listing 2: Starting and enabling Docker

6.3 Step 3: Running OWASP Juice Shop

```
sudo docker pull bkimminich/juice-shop
sudo docker run -d -p 3000:3000 bkimminich/juice-shop
# Verify container is running
sudo docker ps
```

Listing 3: Deploying Juice Shop container

6.4 Step 4: Accessing the Application

Open Firefox and navigate to:

http://127.0.0.1:3000

The OWASP Juice Shop homepage should load, displaying a product listing with a navigation bar including search, account, and basket features.

6.5 Step 5: Burp Suite Configuration

6.5.1 Browser Proxy Settings (Firefox)

- Go to `Settings > Network Settings > Manual Proxy Configuration`
- HTTP Proxy: `127.0.0.1`
- Port: `8080`
- Check: "Also use this proxy for HTTPS"

6.5.2 Burp Suite Intercept Setup

- Open Burp Suite → Proxy → Options
- Ensure listener is on `127.0.0.1:8080`
- Enable "Intercept is on" to capture requests
- Install Burp's CA certificate in Firefox to intercept HTTPS

7 Vulnerability Assessment

7.1 Challenge 1: DOM-Based Cross-Site Scripting (XSS)

7.1.1 Vulnerability Overview

Vulnerability Details

Type:	DOM-Based Cross-Site Scripting (XSS)
Severity:	**High**
OWASP Category:	A03:2021 – Injection
CWE:	CWE-79: Improper Neutralization of Input During Web Page Generation
Challenge:	Perform a DOM XSS attack with iframe `javascript:alert()`

7.1.2 Description

DOM-Based XSS occurs when client-side JavaScript dynamically writes user-supplied data into the Document Object Model (DOM) without proper sanitization or encoding. Unlike Reflected or Stored XSS, the malicious payload never reaches the server — it is processed entirely in the browser.

In OWASP Juice Shop, the search functionality directly reflects user input into the page's DOM without sanitization, making it vulnerable to iframe-based JavaScript injection.

7.1.3 Payload Used

```
<iframe src="javascript:alert(‘xss‘)">
```

Listing 4: DOM XSS Payload

7.1.4 Step-by-Step Procedure

1. Open OWASP Juice Shop at `http://127.0.0.1:3000`
2. Click the search icon in the top navigation bar.
3. In the search box, type or paste the iframe XSS payload:

```
<iframe src="javascript:alert(‘xss‘)">

```

4. Press **Enter** to submit the search query.
5. The browser executes the JavaScript payload and triggers an alert dialog.
6. Juice Shop displays the green success banner confirming the challenge is solved.

7.1.5 Result and Evidence

Challenge Solved: DOM XSS

"You successfully solved a challenge: DOM XSS (Perform a DOM XSS attack with <iframe src='javascript:alert('xss')'>)"

The application confirmed successful execution of the injected JavaScript payload.

The URL after injection appeared as:
`http://127.0.0.1:3000/#/search?q=hello`

This confirms the parameter `q` was not sanitized and the user input was reflected directly into the DOM.

7.1.6 Root Cause Analysis

The Angular-based search component in Juice Shop uses `innerHTML` or unsafe binding to render search results, allowing raw HTML/JavaScript injection without escaping the input. Specifically, the `iframe` with a `javascript:` URI scheme is executed by the browser as a script.

7.1.7 Impact Assessment

Impact Assessment

- **Session Hijacking:** An attacker can steal session cookies using `document.cookie`.
- **Credential Theft:** Fake login forms can be injected to capture user credentials.
- **Defacement:** Application content can be manipulated or replaced entirely.
- **Malware Distribution:** Users can be redirected to malicious external sites.
- **Keylogging:** Keyloggers can be injected to capture user inputs.

7.1.8 Remediation Recommendations

1. Use Angular's built-in **DomSanitizer** to sanitize all user inputs before rendering.
2. Implement a strict **Content Security Policy (CSP)** to block inline scripts and `javascript:` URIs.
3. Encode all output before inserting user-supplied data into the DOM.
4. Avoid using `innerHTML`, `document.write()`, or `eval()` with untrusted data.

7.2 Challenge 2: Bonus Payload XSS

7.2.1 Vulnerability Overview

Vulnerability Details

Type:	Advanced DOM-Based XSS (Bonus Challenge)
Severity:	Medium
OWASP Category:	A03:2021 – Injection
Challenge:	Use the official Juice Shop Bonus Payload in the DOM XSS challenge

7.2.2 Description

This challenge extends the DOM XSS vulnerability by requiring the use of a specific, more complex iframe payload that embeds a SoundCloud music player widget. This demonstrates that XSS can go beyond simple `alert()` dialogs — attackers can embed fully functional external content into victim applications.

7.2.3 Payload Used

```
<iframe width="100%" height="166" scrolling="no" frameborder="no"
allow="autoplay"
src="https://w.soundcloud.com/player/?url=https%3A//api.soundcloud.com/
tracks/771984076&color=%23ff5500&auto_play=true&hide_related=false
&show_comments=true&show_user=true&show_reposts=false
&show_teaser=true"></iframe>
```

Listing 5: Bonus XSS Payload with SoundCloud Embed

7.2.4 Step-by-Step Procedure

1. Open the search functionality in Juice Shop.
2. Paste the full SoundCloud iframe payload into the search box.
3. Submit the search query.
4. The embedded SoundCloud player renders inside the application, and confetti animations appear.
5. Juice Shop confirms the bonus challenge is solved.

7.2.5 Result and Evidence

Challenge Solved: Bonus Payload

"You successfully solved a challenge: Bonus Payload (Use the bonus payload in the DOM XSS challenge.)"

The SoundCloud player successfully embedded within the Juice Shop application, and confetti animations were displayed on screen to celebrate the challenge completion.

7.2.6 Impact Assessment

Impact Assessment

- **Content Injection:** External arbitrary content can be embedded within trusted applications.
- **Phishing:** Attackers can inject fake login forms or notifications.
- **Drive-by Downloads:** Malicious iframes can initiate unintended file downloads.
- **Clickjacking:** Invisible iframes can manipulate user interaction.

7.3 Challenge 3: Broken Authentication (Password Strength)

7.3.1 Vulnerability Overview

Vulnerability Details

Type:	Broken Authentication / Weak Password Policy
Severity:	High
OWASP Category:	A07:2021 – Identification and Authentication Failures
CWE:	CWE-521: Weak Password Requirements
Challenge:	Log in with the administrator's credentials without SQL Injection

7.3.2 Description

This challenge exploits the fact that the administrator account uses a weak, easily guessable password. Despite being an admin account with full system privileges, the credentials can be compromised through simple trial-and-error or common password lists — without requiring any technical injection technique.

This falls under OWASP's **A07:2021 – Identification and Authentication Failures**, which encompasses weak credentials, missing multi-factor authentication, and insecure authentication implementations.

7.3.3 Credentials Discovered

```
Email:    admin@juice-sh.op
Password: admin123
```

Listing 6: Administrator Credentials

7.3.4 Step-by-Step Procedure

1. Navigate to the login page: `http://127.0.0.1:3000/#/login`
2. In the **Email** field, enter: `admin@juice-sh.op`
3. In the **Password** field, attempt common weak passwords.
4. Enter `admin123` as the password and click **Log In**.
5. The application grants administrative access and displays the success challenge banner.
6. The basket shows 6 items, confirming the admin account was accessed with pre-existing data.

7.3.5 Result and Evidence

Challenge Solved: Password Strength

"You successfully solved a challenge: Password Strength (Log in with the administrator's user credentials without previously changing them or applying SQL Injection.)"

Full administrative access was granted using the weak default password `admin123`.

7.3.6 Root Cause Analysis

The administrator account was configured with an extremely weak password (`admin123`) that:

- Appears in common password dictionaries (rockyou.txt, etc.)
- Does not meet minimum complexity requirements
- Was never rotated from its default value
- Would be cracked instantly by any brute-force or dictionary attack tool

7.3.7 Impact Assessment

Impact Assessment

- **Full Administrative Access:** Complete control over the application and all user data.
- **Privilege Escalation:** An attacker gains the highest level of system privilege.
- **Data Breach:** All customer orders, emails, and personal data become accessible.
- **Account Manipulation:** Ability to modify, delete, or create any user account.
- **Financial Fraud:** Access to order management and payment processing.

7.3.8 Remediation Recommendations

1. Enforce a **strong password policy**: minimum 12 characters, uppercase, lowercase, digits, and symbols.
2. Implement **Multi-Factor Authentication (MFA)** for all privileged accounts.
3. Use **account lockout** after 5 consecutive failed login attempts.
4. Conduct **regular password audits** and force resets of weak credentials.
5. Deploy **Have I Been Pwned** integration to block known compromised passwords.

7.4 Challenge 4: Information Disclosure (Privacy Policy)

7.4.1 Vulnerability Overview

Vulnerability Details

Type:	Information Disclosure / Sensitive Data Exposure
Severity:	Medium
OWASP Category:	A02:2021 – Cryptographic Failures / A01:2021 – Broken Access Control
CWE:	CWE-200: Exposure of Sensitive Information
Challenge:	Read the privacy policy of Juice Shop

7.4.2 Description

This challenge demonstrates how applications often expose sensitive or useful reconnaissance information through publicly accessible pages, especially privacy policies, terms of service, and about pages. While these pages are intended for legal disclosure, attackers can extract valuable intelligence about the application's infrastructure, data handling, and internal systems.

7.4.3 Location Accessed

`http://127.0.0.1:3000/#/privacy-security/privacy-policy`

7.4.4 Step-by-Step Procedure

1. Navigate to the Account menu or footer links.
2. Find and access the Privacy Policy page.
3. Read through the policy — the challenge is completed upon visiting the page.
4. Juice Shop confirms the challenge is solved with the green success banner.
5. The privacy policy reveals internal URLs (`http://127.0.0.1`), effective dates, and data collection practices.

7.4.5 Information Exposed

The Privacy Policy page revealed:

- **Internal IP Address:** `http://127.0.0.1` referenced multiple times as the service URL.
- **Effective Date:** March 15, 2019 — indicating the software version era.
- **Data Types Collected:** Email, address, city, ZIP code, cookies, usage data, tracking technologies.
- **Third-party Tools:** Use of beacons, tags, and analytics scripts revealed.

7.4.6 Result and Evidence

Challenge Solved: Privacy Policy

"You successfully solved a challenge: Privacy Policy (Read our privacy policy.)"

Accessing the privacy policy page triggered the challenge completion and revealed internal infrastructure information that assists further reconnaissance.

7.4.7 Impact Assessment

Impact Assessment

- **Reconnaissance Assistance:** Exposed internal URLs, IP addresses, and application stack details.
- **Social Engineering:** Data collection practices can be weaponized for targeted phishing.
- **Increased Attack Surface:** Knowledge of tracking scripts and third-party dependencies reveals potential supply-chain attack vectors.
- **Regulatory Risk:** Outdated privacy policy may indicate GDPR/compliance gaps.

7.4.8 Remediation Recommendations

1. Remove all internal IP addresses and server-side references from public-facing pages.
2. Regularly review and update privacy policies to avoid leaking architecture details.
3. Replace internal URLs with production domain names in all customer-facing content.
4. Conduct regular content audits to identify unintended information disclosure.

7.5 Challenge 5: Business Logic Vulnerability (Bully Chatbot)

7.5.1 Vulnerability Overview

Vulnerability Details

Type:	Business Logic Vulnerability
Severity:	Medium
OWASP Category:	A04:2021 – Insecure Design
CWE:	CWE-840: Business Logic Errors
Challenge:	Receive a coupon code from the support chatbot

7.5.2 Description

Business logic vulnerabilities arise when an application's workflow can be manipulated to perform actions outside its intended design. In this challenge, the Juice Shop support chatbot (*juicy-chat-bot*) can be socially engineered through persistent or demanding messages into revealing a coupon code — a resource it should only provide under legitimate conditions.

This is a classic example of insecure design where the application's conversational AI does not implement proper authorization checks before releasing privileged information.

7.5.3 Step-by-Step Procedure

1. Navigate to the Support Chat at: `http://127.0.0.1:3000/#/chatbot`
2. The chatbot greets the user: *"Nice to meet you, hi, I'm Juicy"*
3. Send a demanding or persistent message to the bot. Example message used:

   "heyy i want delivery in one day but i don't have membership"

4. The chatbot, unable to handle the persistent request appropriately, responds:

   "Oooookay, if you promise to stop nagging me here's a 10% coupon code for you: ***k#pDmhz3Tq****"*

5. Juice Shop confirms the "Bully Chatbot" challenge is solved.

7.5.4 Result and Evidence

Challenge Solved: Bully Chatbot

"You successfully solved a challenge: Bully Chatbot (Receive a coupon code from the support chatbot.)"

A 10% discount coupon code (`k#pDmhz3Tq`) was obtained by manipulating the chatbot's response logic through persistent messaging.

7.5.5 Root Cause Analysis

The chatbot's conversational logic contains a fallback condition — when it cannot understand or adequately respond to a user's request after multiple attempts, it offers a coupon code as a "resolution." This represents a fundamental design flaw:

- No authorization check before issuing coupon codes.
- The chatbot's decision tree exposes a privileged action through a social engineering path.
- No rate limiting or anomaly detection on repeated/aggressive messages.

7.5.6 Impact Assessment

Impact Assessment

- **Financial Loss:** Unauthorized discount codes can be exploited at scale, causing direct revenue loss.
- **Abuse of Functionality:** Any user can obtain discounts without meeting eligibility criteria.
- **Scalability Risk:** Automated bots could send thousands of nagging messages to harvest coupon codes.
- **Reputation Damage:** Exploitation at scale could devalue the coupon program.

7.5.7 Remediation Recommendations

1. Implement **strict authorization checks** before any privileged action (coupon issuance) in chatbot flows.
2. Add **rate limiting** on chatbot interactions per user per session.
3. Review and redesign the **chatbot decision tree** to eliminate all unauthorized disclosure paths.
4. Log and monitor chatbot interactions for anomalous patterns.
5. Replace hardcoded fallback responses with proper escalation to a human agent.

8 HTTP Request Analysis with Burp Suite

Burp Suite was used throughout the testing to intercept, analyze, and replay HTTP traffic between Firefox and the Juice Shop application. Key observations include:

8.1 Traffic Captured

Request Type	Endpoint	Security Finding
POST	`/rest/user/login`	Credentials sent in plaintext JSON body
GET	`/api/Products/?q=`	Search input not sanitized, enabling XSS attacks
GET	`/#/privacy-security/privacy-policy`	Sensitive application details exposed publicly
POST	`/rest/chatbot/respond`	No authentication validation before coupon issuance
GET	`/rest/user/whoami`	JWT token visible in Authorization header

8.2 Key Observations

- **Authentication Tokens:** JWT tokens were transmitted in Authorization headers; these can be decoded to expose user claims.
- **CORS Headers:** Permissive CORS configuration observed — cross-origin requests allowed broadly.
- **Error Messages:** Verbose error responses from the API exposed internal stack traces in some cases.
- **Cookie Flags:** Session cookies lacked `HttpOnly` and `Secure` flags in some instances.

9 Challenges Faced

During the four-week engagement, the following challenges were encountered and resolved:

1. **Burp Suite Certificate Installation:** Initial difficulty intercepting HTTPS traffic until the Burp CA certificate was properly installed in Firefox's certificate store.
2. **Payload Identification for XSS:** Determining the correct iframe payload format required understanding Angular's DOM rendering behavior.
3. **Chatbot Interaction Logic:** Multiple message patterns were tested before finding the trigger phrase that caused the chatbot to release the coupon code.
4. **Browser Proxy Configuration:** Ensuring Firefox exclusively used the Burp proxy without falling back to system settings required disabling automatic proxy detection.
5. **Docker Networking:** Initial container binding issues were resolved by explicitly mapping port 3000 to localhost.

10 Skills Gained

This project provided practical exposure to the following technical skills:

Skill Area	Specifics
Penetration Testing	Web application attack methodology, challenge-based testing
Burp Suite	Proxy setup, request interception, HTTP traffic analysis, Repeater usage
OWASP Top 10	Practical understanding of A01–A10 vulnerability categories
XSS Attacks	DOM-based XSS, iframe injection, browser execution context
Authentication Testing	Weak password identification, admin credential attacks
Business Logic Testing	Chatbot manipulation, workflow abuse
Docker	Container deployment, port mapping, service management
Linux (Kali)	Command-line tools, service management, network configuration
Security Documentation	Professional pentest report writing, evidence capture

11 Conclusion

This four-week penetration testing project on OWASP Juice Shop provided comprehensive, hands-on exposure to real-world web application security vulnerabilities. All five target challenges were successfully identified and exploited:

1. **DOM-Based XSS** demonstrated the critical risk of unsanitized client-side input processing.
2. **Bonus Payload XSS** extended the attack to show how complex content can be injected into applications.
3. **Broken Authentication** highlighted the severe consequences of weak administrative credentials.
4. **Information Disclosure** showed how even public pages can leak sensitive reconnaissance data.
5. **Business Logic Vulnerability** revealed how conversational AI systems can be manipulated to bypass authorization controls.

The structured week-by-week approach — from environment setup through exploitation to documentation — provided a realistic simulation of a professional penetration testing engagement. The use of Burp Suite for traffic analysis, combined with manual exploitation techniques, deepened understanding of both offensive and defensive web security practices.

12 Future Scope

Building on this project, the following areas are planned for future exploration:

- **SQL Injection Testing:** Automated and manual SQLi attacks using sqlmap and manual payloads.
- **Cross-Site Request Forgery (CSRF):** State-changing request forgery against authenticated sessions.
- **Automated Scanning:** OWASP ZAP full application scan and vulnerability report generation.
- **API Security Testing:** JWT manipulation, IDOR attacks on REST endpoints.
- **Advanced Burp Suite:** Extensions such as Active Scan++, Logger++, and Turbo Intruder.
- **Secure Linux Server Setup:** Hardening a Linux server with fail2ban, auditd, and CIS benchmarks (Project 3).

13 References

1. OWASP Official Website: https://owasp.org
2. OWASP Juice Shop Project: https://owasp.org/www-project-juice-shop/
3. OWASP Top 10 2021: https://owasp.org/www-project-top-ten/
4. Burp Suite Documentation: https://portswigger.net/burp/documentation
5. OWASP Juice Shop GitHub: https://github.com/juice-shop/juice-shop
6. CWE Mitre Database: https://cwe.mitre.org
7. Docker Documentation: https://docs.docker.com

Appendix A: Challenge Completion Summary

No.	Challenge Name	Status	Week Completed
1	DOM XSS	**SOLVED**	Week 2
2	Bonus Payload XSS	**SOLVED**	Week 2
3	Password Strength (Admin Login)	**SOLVED**	Week 3
4	Privacy Policy (Info Disclosure)	**SOLVED**	Week 3
5	Bully Chatbot (Business Logic)	**SOLVED**	Week 3

Appendix B: OWASP Top 10 Coverage Matrix

OWASP ID	Category	Tested
A01:2021	Broken Access Control	Partially
A02:2021	Cryptographic Failures	
A03:2021	Injection (XSS)	
A04:2021	Insecure Design	
A05:2021	Security Misconfiguration	Partially
A06:2021	Vulnerable Components	Future
A07:2021	Auth Failures	
A08:2021	Software & Data Integrity	Future
A09:2021	Security Logging Failures	Observed
A10:2021	Server-Side Request Forgery	Future